

Chapter 1

Introduction to Concurrency

1.1 Abstract Concurrency Concepts

The chapter starts by taking a helicopter view on concurrency where we ignore the details of a concrete programming language. Instead, the focus is qualitative, identifying a few core concepts that are fundamental when programming for concurrency.

Programming for concurrency is based on a number of concepts and abstractions used for writing software with multiple independent streams of statements/instructions.¹ This may be contrasted with sequential programming which deals with only a single stream of statements/instructions. We use the term *streams of statements/instructions* or simply *stream* to denote software component with a single (possibly unbounded) sequence of statements, e.g.,

```
stream t = {  
    s1;s2;s3; ...  
};
```

The body of a stream may contain all the well-known programming concepts e.g. loops, if-statements, variable declarations, functions etc. Almost anything, one would write in the `main` method of a Java program.

The syntax above close to Java's syntax, but the some of the code in this section is not valid Java.

In this section, we consider concurrency as an abstraction that has a number of concrete manifestations in programming languages, libraries, protocols and many other places including human interaction. This is similar to other abstractions e.g. an animal which has a number of properties common to a lot of living beings (a cat, a spider, a hummingbird, etc.). Such abstractions are a common tool in almost all sciences (and many other places).

Programming languages for concurrency have not yet settled on a common/standardized set of concepts. This is in contrast to programming concepts such as functions/methods and parameters which are roughly the same in most programming languages. In the section "Concurrency concepts" we present and discuss some of the many ways streams and stream coordination are realized in various programming languages. Except for that section, we focus on Java in these notes.

¹The term stream of statements implies that there is an ordering in the execution of the statements and also that the stream can be composed of an unbounded number of statement. In Java there is an interface `Stream` which has nothing to do with our use of the term stream in this chapter.

Abstract:	streams: $s_1; s_2; s_3; \dots$ $z_1; z_2; z_3; z_4; \dots$
Concrete:	Java threads / callbacks / processes / tasks / coroutines / ...

Figure 1.1: Streams of statements are an abstraction that denotes (possibly unbounded) and ordered sequences of statements that are executed independently. This abstraction is implemented (or made concrete) in different ways for different programming languages, frameworks, libraries, etc.

In the rest of this section we will use the term *concurrent program* for code that have two or more independent streams.

1.2 Classification of concurrency

Most often concurrency concepts are discussed with a strong emphasis on performance. This is natural because concurrency is often utilized to improve performance. However, the performance bias often introduce extra complexity. For example, the interplay of the memory model and locking mechanisms in Java is quite complex. In this section, we focus on introducing a few abstract concurrency concepts ignoring the complexity of their efficient implementation.

The concept of stream of statements (introduced above) is used as an abstraction for discussing the following three fundamental motivations for concurrency:

- User interfaces and other kinds of input/output (*Inherent*).
- Hardware capable of simultaneously executing multiple streams of statements (*Exploitation*), a special (but important) case is communication and coordination of independent computers on the internet.
- Enabling several programs to share some resources in a manner where each can act as if they had sole ownership (*Hidden*).

These terms were coined by the Norwegian computer scientist Kristen Nygaard (https://en.wikipedia.org/wiki/Kristen_Nygaard).

Concurrency programming has a number of additional challenges compared to sequential programming; first and foremost, handling *coordination* when the independent streams need to collaborate, compete for resources or exchange information. The term *coordination* is used here as an abstraction of more low-level terms like synchronization, locking, remote procedure calls, message passing ... that one typically finds in writings about concurrency.

Concurrency and *nondeterminism* are two related but different concepts. Concurrent programs may behave nondeterministically, for example, when different streams share variables, and the result of the computation depends on the detailed timing of the statements in the streams. However, a nondeterministic computation may be completely sequential e.g. if a statement for throwing a dice (coin, random number, ...) is introduced.

The greatest challenge in concurrency programming is getting the streams to coordinate reliably in the face of this nondeterminism. Therefore, it is important not to prematurely focus on performance. This will only complicate getting the coordination correct.

1.2.1 A brief history of concurrency and terminology

The very first computers for general purpose were only meant for running one program (stream) at a time. However, they gradually became powerful enough that a small number of users could share the computer. The computer itself could still only execute a single sequence of commands/instructions. However, users had the illusion that they had the computer for themselves, because the computer switched its attention between the users. This was called *timesharing* (analogous to the concept of timesharing an apartment) and addressed the exploitation motivations. Today, we have the somewhat opposite situation where the hardware in a laptop or smart phone is capable of running several streams simultaneously, because it has a small number of processors called *cores* that can each run a stream.

To write the software for supporting timesharing (exploitation) the first concurrency programming concepts evolved. They made it possible to write programs with several streams (at that time called tasks or processes). The streams also needed mechanisms for coordinating their computations e.g. to access external devices (such as the printer) or exchange data. Almost immediately, these mechanisms came in two different flavors: message passing and shared memory.

The focus of concurrency programming has shifted several times since the early days of timesharing. For example, the use of computers for simulation led to the programming language SIMULA67. Simulations typically have a (large) number of streams each representing some activity in the real world: people, physical phenomenon like the weather, productions lines in a factory etc. SIMULA67 has a stream concept used for describing such activities: co-routines. This concept has since been re-invented several times e.g. in the Kotlin programming language that is now the primary language for Android development.

Very early, computers became connected in a network and used for telecommunication. The interaction between the computers was achieved via. telecommunication lines and hence message passing was an obvious coordination mechanism. This led to programming languages with constructs for sending and receiving messages. The Internet protocols introduced at that time are also based on message passing. These (message-passing) protocols are still the foundation of the Internet.

Over the years, many programming languages supporting concurrency have been introduced often reinventing the fundamental concepts: streams of statements and coordination.

1.2.2 A brief note on hardware details relevant for concurrency

Modern hardware have several features that supports and exploits concurrency. First and foremost the CPU may have several processors that can execute code independently of each other. These processors are most often called cores. The CPU of a modern laptop typically have from 4 (and upwards) such cores. This is how you may detect the number of cores on your computer:

- *Linux*: `nproc`
- *Windows*: Open task manager (Ctrl + Shift + Esc) and select Performance tab
- *Mac*: Hit Command+Spacebar to open Spotlight, then type “System Information” and hit return (or type “System Profiler” for earlier MacOS versions) Click “Hardware” to see the Hardware Overview, and find the “Total Number of Cores”

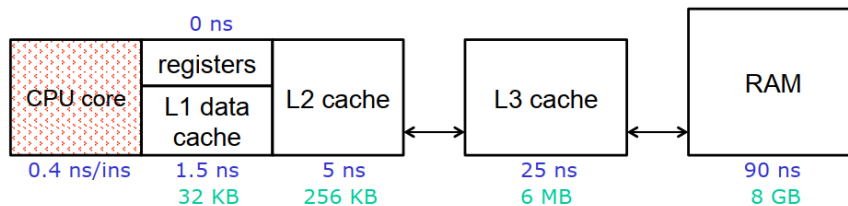


Figure 1.2: Memory hierarchy example. Copied from A multicore performance mystery solved by Peter Sestoft (sestoft@itu.dk)

Most of the material in this lecture note focuses on how to take advantage of such multi-core CPU's. However, the organization of the computers memory may also influence the running time of your code.

Memory hierarchy

Figure 1.2 illustrates the memory components (caches) of an Intel i7-4870HQ CPU. The L1 and L2 caches are local to a core; the L3 cache and RAM are shared between cores.

The numbers (in blue) indicates the time it takes to fetch (or store) data in the different parts of the memory hierarchy. These numbers illustrate that it is more than an order of magnitude faster to reach memory in the local memory of a core compared to reaching memory in the RAM.

The disadvantage of having data in the local memory of a core is that changes are not visible from other cores.

Fortunately, programming languages include mechanisms to control where variables are placed in the memory hierarchy. For example, in Java, the modifier `volatile` ensures (among other things) that variables are stored in memory shared by all cores (and hence variable writes performed by one core are visible by the other cores).

In later chapters, we explore in more depth how hardware impacts performance in concurrent software, how to effectively measure runtime and how to exploit available hardware.

1.2.3 Going into more details with the three types of concurrency.

The three fundamental types of concurrency described above are not always viewed as instances of a more general concept. The thesis of this section is *that there are some common abstractions covering all three*. Performance details are set aside (because they often introduce a lot of low-level considerations). Performance may of course turn out to be important, but this is certainly not always the key challenge. Finding the right abstract program structure is often the key to writing good software. Historically, the importance of the fundamental motivations for concurrency has changed over time, and hence a number of programming language constructs introduced over the years have only addressed one or two of them well.

Motivation 1 (Exploitation concurrency) – Hardware capable of simultaneously executing multiple streams of statements

Today, most laptops, smart phones and servers have hardware capable of executing several streams of instructions simultaneously. These are called multi-core computers, and each core is an independent processor. To take advantage of multiple cores the software must be written in such a way that independent streams of statements can be separated and directed to the different cores/processors.

Below is an example of pseudo-code where multiple concurrent streams are used to speed up a computation (running on a computer with multiple cores). Three streams collaborate on counting the number of primes less than 3 million:

```
stream t1= {
    for (int i=0; i<999999; i++)
        if (isPrime(i)) counter.increment();
};

stream t2= {
    for (int i=1000000; i<1999999; i++)
        if (isPrime(i)) counter.increment();
};

stream t3= {
    for (int i=2000000; i<2999999; i++)
        if (isPrime(i)) counter.increment();
};
```

Motivation 2 (Inherent concurrency) – Interacting with the environment

Almost any computer needs to communicate with its environment, e.g. to get input or to show results. In sequential programs this may be done with statements like `read` or `write`:

```
n= read();
res= computeNoOfPrimes(n);
...
```

However, this will force the computer executing the code to stop when reaching the `read()` and wait for the user to provide input. This is not convenient, if there are many independent communication channels e.g. multiple buttons and text fields on a webpage, or on a smart phone. Similarly, for a computer embedded in a robot that needs to listen for input from many different sensors to navigate safely.

So, whenever there is *an independent stream of external events* that needs the computer's attention, there *should be an independent stream in its program* handling the stream of events.

To illustrate inherent concurrency, consider an it-system to keep track of the number of visitors to Tivoli. At each entrance there is a turnstile:

Each of these turnstiles contains some electronics (probably a small computer) that signals whenever a visitor goes through the turnstile.

```
stream turnStile1= {
    await(visitor); // signalling a visitor at turnstile 1
    visitors1= visitors1 + 1;
```

```

//e-mail app
stream Email= {
  do {
    await(email);
    notify(user);
    store(email_in_inbox);
  } forever
}

// Stopwatch app
stream stopWatch= {
  do {
    alarmAt= await(start button);
    waitUntil(alarmAt);
    notify user;
  } forever
}

```

Listing 1.1: Pseudo-code modeling two streams of statements for an e-mail app and a stopwatch app executed concurrent.

```

};

stream turnStile2= {
  await(visitor); // signalling a visitor at turnstile 2
  visitors2= visitors2 + 1;
};

stream turnStile3= {
  await(visitor); // signalling a visitor at turnstile 3
  visitors3= visitors3 + 1;
};

```

The details of how the value of the three counters are communicated to the central computer is not important here (however we return to this a later in the chapter on message passing). The key point is that the arrival of visitors to the three turnstiles are independent events and hence require a separate stream.

Motivation 3 (Hidden concurrency) – Resource sharing

Many applications may be active on a smartphone or computer, but most of the time they are idle waiting for something to happen e.g. an e-mail arriving, download finishing, alarm clock activated etc. Therefore, the underlying operating system will typically have many streams executing on the same processor (core). Since each of the streams is idle (waiting) most of the time, none of them will “notice” that other streams are running on the same processor while they are waiting. Listing 1.1 illustrates hidden concurrency for an e-mail app and stopwatch app running concurrently.

```
public class LoggerThread extends Thread {
    List<Integer> logbook;
    Integer visitorID;

    public LoggerThread(List<Integer> logbook,
                        Integer visitorID) {
        this.logbook = logbook;
        this.visitorID = visitorID;
    }

    @Override
    public void run() {
        logbook.add(visitorID);
    }
}
```

Listing 1.2: Example of a visit logging thread.

1.3 Java Threads

Java concurrent programs are composed by independent units of sequential computation known as *threads*. This is the Java implementation of abstract concept of streams of states/instructions that we discussed in sections 1.1 and 1.2. Precisely, we define a thread as follows.

Definition 1 (Thread). A *thread* is an independent sequence of program statements.

In the definition, “independent” means that each thread has its own flow of execution. As we will see, threads can communicate and coordinate, but each thread has its own flow of execution. The term “sequence” refers to order of execution. In principle, operations are issued to the CPU in the order in which they appear in the thread declaration. However, hardware and runtime factors may alter the visible order of execution; we will discuss this details in chapter 4.

1.3.1 Defining a thread

A common way to define a thread in Java is as a class that extends from `Thread` and overrides the `run()` method with the computation that the thread must execute. Listing 1.2 shows an example of thread definition. The `LoggerThread` is executed every time a Tivoli visitor crosses one of its turnstiles.

The computation that the thread carries out is defined in the `run()` method. This thread adds a visitor ID to the log book.

The example also shows that thread parameters can be defined as fields in the thread class. We use the fields `logbook` and `visitorID` as parameters that the thread uses while it is running.

1.3.2 Starting a thread

To start a thread, we must create an instance of the thread class and use the `start()` method. In concurrent Java applications, it is common to define a main entry point that starts the

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        List<Integer> logbook = new ArrayList<Integer>();
        LoggerThread thread1 = new LoggerThread(logbook, 1);
        thread1.start();
        thread1.join();
        System.out.println(logbook.toString());
    }
}
```

Listing 1.4: Starting a `LoggerThread` and waiting until it finishes its execution.

threads in the application. In fact, the entry point of a Java application is executed by a “main” thread. For sequential Java applications, this is the only thread in the program, which is why they do not contain concurrent behavior.

The code below illustrates how to start a `LoggerThread` from a `Main` class. The `Main` class is the entry point for the Java application.

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> logbook = new ArrayList<Integer>();
        LoggerThread thread1 = new LoggerThread(logbook, 1);
        thread1.start();
    }
}
```

Listing 1.3: Starting a `LoggerThread`.

The code defines a `Main` class with the standard entry point method `main(...)` in Java applications. First, we create a `logbook` that the `LoggerThread` will use to add the entry. Then, we create an instance `thread1` of `LoggerThread` with a reference to the `logbook` and the visitor id 1. At this point, no code in `thread1` has been executed yet. The statement `thread1.start()` starts the execution of the thread. It is important to note that `start()` is a non-blocking call, i.e., the main thread starts the execution of the thread but it does not wait until it finishes.

1.3.3 Waiting for a thread to finish execution

We can inspect the state of the `logbook` to observe that `thread1` has added the entry to list. However, we must wait for `thread1` to finish its execution; as otherwise we may read the state of `logbook` before the element is added. To this end, we use the `join()` function.

In listing 1.4, the program statement `thread1.join()` blocks the main thread until `thread1` has finished its execution. That is, it waits until all the operations in the `run()` method in `LoggerThread` have been executed. As a consequence, the last `println(...)` will show the state of `logbook` after the `thread1` has added the element to the list.

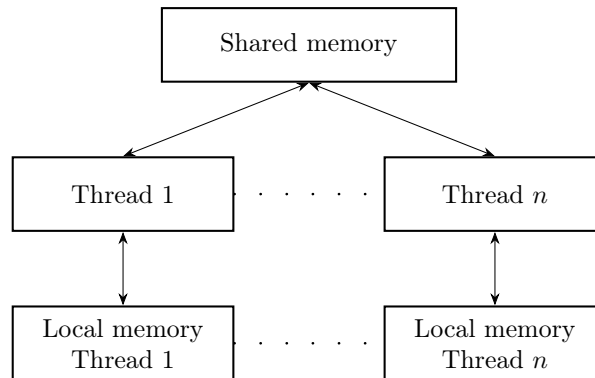


Figure 1.3: Illustration of types of memory accessible to threads

Note that we have added `throws InterruptedException` to the signature of the `main(...)` method. This is because the `join()` function requires to handle this type of exception. When a thread is blocked (or waiting), it might be interrupted. If the thread is interrupted, then it will restart execution and throw an `InterruptedException`. You may use a `try-catch` statement to handle the exception; the code above omits it for simplicity.

1.3.4 Shared vs local memory in threads

Java threads can access shared and local memory. Figure 1.3 illustrates the direction of communication. Local memory is only accessible within a thread, e.g., in fig. 1.3 thread 1 can only access local memory 1. Shared memory can be accessed by all threads. Shared memory is the main communication medium for threads.

Listing 1.5 shows an example where two threads access the shared variable `logbook`. Variable `logbook` is shared memory because the threads get a reference to the object during initialization; i.e., call-by-reference semantics. The field `visitorID` in `LoggerThread` is an example of local memory. Since `visitorID` is an integer, the call to the constructor makes a copy of the value that is only locally accessible to the thread; i.e., it uses call-by-value semantics. Generally, a variable is shared if more than one thread has access to a reference to the object.

1.4 Java scheduler

The execution of threads is *non-deterministic*. The Java Virtual Machine (JVM) decides when a thread is allowed to use the CPU to execute operations. The part of the JVM responsible for this task is known as the Java *scheduler*.

To understand how the scheduler works, we first introduce a simplified model of thread states.² The model is depicted in fig. 1.4. When a thread starts, it goes into the *enabled* state. In this state, all information necessary to execute the thread is in place (e.g., memory has been allocated for the thread to execute), but it is not executing any code. The scheduler selects non-deterministically threads in the enabled state and moves them into the *running* state. The scheduler also selects threads in the running state and send the back to the enabled state after

²For a more detailed model of the states of a thread see chapter 5.

```

import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        List<Integer> logbook = new ArrayList<Integer>();
        LoggerThread thread1 = new LoggerThread(logbook, 1);
        LoggerThread thread2 = new LoggerThread(logbook, 2);
        thread1.start();
        thread2.start();
        thread1.join();
        thread2.join();
        System.out.println(logbook.toString());
    }
}

```

Listing 1.5: Starting two `LoggerThread` that share access to `logbook`.

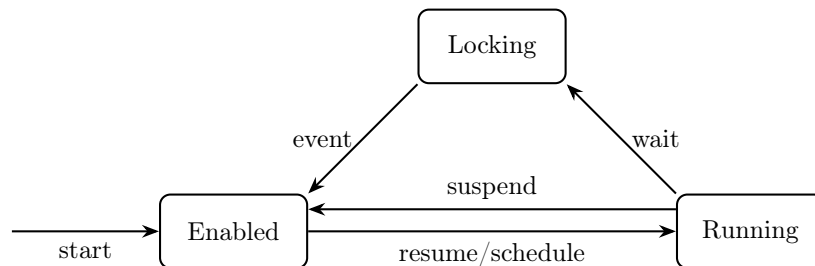


Figure 1.4: Simplified thread state model.

they have been running for some time. In this way, even computers with one CPU can execute multiple tasks at the same time. Alternatively, a thread may move to the *locking* state. A thread transitions to the locking state if it cannot continue execution until some event (not controlled by the thread) occurs. For instance, when the main thread in 1.5 executes `join()`, it moves to the locking state, as it must wait for a thread to finish. When the thread finishes, it sends a signal to main, which returns to the enabled state to continue execution when the scheduler decides.

1.5 Interleavings

Due to the non-deterministic nature of concurrent programs, the same concurrent program can produce different results if executed repeatedly.

For example, consider again the program in listing 1.5. Think about the following question: what are the possible values that this program will print?

An obvious output is `[1,2]`, which results from `thread1` adding visitor 1 to `logbook` and then `thread2` adding visitor 2. However, the opposite can also occur, namely, 2 is added first and then 1, which results in the output `[2,1]`.

Perhaps surprisingly, the program can also print just `[1]` or just `[2]`. This is because

the `add(...)` method that `LoggerThread` executes is not *atomic*; i.e., it is not executed as an indivisible operation.

Internally, an `ArrayList` (like `logbook`) uses the number of elements in the list as a pointer to a location to store the element to add. If, for instance, `thread2` starts adding an element before `thread1` has updated the size of the list, then both threads will add the element in the same location. This would replace the element added by `thread1` with that of `thread2`; instead of adding both elements.

Generally, to reason about the behavior of concurrent programs we must analyze the possible *interleavings* of operations.

Definition 2 (Interlaving). An *interleaving* is a possible sequence of operations for a concurrent program.

Syntax for interleavings In order to have a common language for interleavings, we use the following syntax to denote that a thread `<thread>` executed operation `<step>`:

`<thread>(<step>)`

For example, the interleaving that produces the output `[1,2]` would be written as

```
main(List<Integer> logbook = new ArrayList<Integer>());
main(LoggerThread thread1 = new LoggerThread(logbook, 1));
main(LoggerThread thread2 = new LoggerThread(logbook, 2));
main(thread1.start());
main(thread2.start());
thread1(logbook.add(1));
thread2(logbook.add(2));
main(thread1.join());
main(thread2.join());
main(System.out.println(logbook.toString()))
```

Depending on the purpose of the interleaving we may use the syntax `...` to omit operations that are not relevant to our goal. For instance, to show that the interleaving above prints `[1,2]` the most important detail is the execution of the `add()` instructions. So, we could write the interleaving as follows:

```
...; thread1(logbook.add(1)); thread2(logbook.add(2)); ...
```

The level of granularity of the interleaving is also important. Clearly, not all the operations mentioned in the interleavings above are atomic. Sometimes, it is required to increase the level of granularity of an interleaving to describe the behavior of the program.

For instance, consider the task of describing an interleaving showing that `[1]` is a possible output of the program in listing 1.5. To this end, we must look into the implementation of the `add()` method in `ArrayList`; because considering `add()` as an atomic statement would never lead to the output `[1]`. Listing 1.6 shows the relevant parts of the implementation of the `add()` method. The `public` method `add(E e)` is called when executing `logbook.add(...)`, and the `private` method `add(E e, Object[] elementData, int s)` is called by the `public` method. The field `elementData` and `size` are fields of the `ArrayList` class, which contain the elements of the `ArrayList` and its size, respectively. The following interleaving produces the output `[1]`:

```
...; thread1(elementData[s]=e); thread2(elementData[s]=e); thread1(size=s+1);...
```

```

...

private void add(E e, Object[] elementData, int s) {
    if (s == elementData.length)
        elementData = grow();
    elementData[s] = e;
    size = s + 1;
}

...

public boolean add(E e) {
    modCount++;
    add(e, elementData, size);
    return true;
}

...

```

Listing 1.6: Internal implementation of the `add()` method in `ArrayList`; relevant to explain the interleaving that outputs [1]. Source: <https://github.com/openjdk/jdk/blob/3d47518697e794bf4afb577f16afb278fbf3d30a/src/java.base/share/classes/java/util/ArrayList.java>

The issue is that `thread2` writes on the same position `s` as `thread1` because it executes the write before `thread1` updates `size=s+1`.

This is not the lowest level of granularity. Many single line program statements such as `size = s + 1` are compiled into multiple machine level instructions. Thus, in order to discuss some concurrent issues, one may need to look into finer granularity of operations. We will discuss lower level concurrency issues in later chapters.

1.6 Race conditions

Producing different results depending on interleavings is a well-known phenomenon in concurrency programming. It is known as a *race condition*.

Definition 3. A *race condition* occurs when the result of the computation depends on the interleavings of the operations.

Generally, race conditions are undesirable. They may produce program outputs that are incorrect, as we have seen in the previous section.

However, not all race conditions lead to undesired behavior. For instance, it would be fine if the only two outputs of the program in listing 1.5 were [1,2] and [2,1]. That is, it is perfectly fine that visitor 2 enters before visitor 1 or vice versa. Although this means that the program contains race conditions; as two possible outputs are possible depending on interleavings. However, what we do not want is to miss logging a visitor entering Tivoli.

1.7 Mutual Exclusion

The concurrency issues that we have seen in previous sections all arise from the fact that multiple threads are accessing shared memory in a manner that produces undesired behavior. A possible solution to these problem is to prevent that multiple threads access shared memory concurrently. This is one of the first problems that arose in the field of concurrent programming, and it is known as the *mutual exclusion problem*.

Before precisely describing mutual exclusion, we introduce the notion of critical section.

Definition 4 (Critical section). A *critical section* is a part of the concurrent program where concurrent access must be restricted.

By restricted access we mean, e.g., that at most one thread at a time can execute the operations in the critical section or that only threads that read shared memory are allowed to access the critical section. In the context of mutual exclusion we focus on the former, but in later chapters we will see that different restrictions may apply to critical sections depending on the concurrency problem at hand.

For the program in listing 1.5, the critical section would be all the code that is executed in the method `logbook.add()`.

Now we are ready to define mutual exclusion:

Definition 5 (Mutual Exclusion). The *mutual exclusion* property states that two or more threads cannot execute their critical section at the same time.

Again in our example, and given the critical section defined above, the reason why observe the outputs [1] or [2] is due to the lack of mutual exclusion.

1.7.1 Enforcing mutual exclusion via Locks

Many methods exist to enforce mutual exclusion. In this section, we focus on the `Lock` interface provided by Java standard library. This is one of the simplest and most widely used mechanisms to ensure mutual exclusion—we will explain other mechanisms in later chapters. Also, here we only look into how to use locks, but in chapter 5 we describe in depth how to implement them.

Definition 6 (Lock). A *lock* is a data structure with a Boolean state indicating whether the lock is available or not. Locks have two main methods with the following behavior:

- `lock()` - If the lock is available, it sets it to unavailable and the thread executing the call continues execution. If the lock is unavailable, the thread blocks (i.e., it goes into the locking state).
- `unlock()` - If there are any blocked threads waiting for the lock, then it signals one of them (and it moves to the enabled state). If there are no blocked threads, it sets the lock to available. In either case, the thread executing the call continues execution.

Java defines the `Lock` interface with the methods above. The most commonly used implementation of this interface is the `ReentrantLock` class. The behavior of lock and unlock is the same as described above, but it allows threads to acquire the same lock multiple times (thus, the re-entrant property).

```

public class LoggerThread extends Thread {
    List<Integer> logbook;
    Integer visitorID;
    Lock lock;

    public LoggerThread(List<Integer> logbook, Integer visitorID, Lock lock) {
        this.logbook = logbook;
        this.visitorID = visitorID;
        this.lock = lock;
    }

    @Override
    public void run() {
        lock.lock()
        try{
            logbook.add(visitorID);
        } finally {
            lock.unlock();
        }
    }
}

```

Listing 1.7: Logging thread ensuring mutual exclusion when logging a visit.

```

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        List<Integer> logbook = new ArrayList<Integer>();
        Lock lock = new ReentrantLock();
        LoggerThread thread1 = new LoggerThread(logbook, 1, lock);
        LoggerThread thread2 = new LoggerThread(logbook, 2, lock);
        thread1.start(); thread2.start();
        thread1.join(); thread2.join();
        System.out.println(logbook.toString());
    }
}

```

Listing 1.8: Starting two `LoggerThread` that use a lock to access `logbook`.

A possible way to ensure mutual exclusion on `logbook.add()` is to require `LoggerThread` class acquire a lock before adding the visitor to the logbook, and releasing the lock after the operation is completed. Listings 1.7 and 1.8 show an implementation. However, as we will see in later chapters, a better option is to extend data structures (such as `ArrayList`) with locks so that multiple threads can access them concurrently.

1.7.2 A good solution to the mutual exclusion problem

Generally, after you have identified the critical section in your program, you can always ensure mutual exclusion by forcing all threads to acquire a lock before entering the critical section and releasing it after all operations have completed. While this looks like a very simple recipe, it can lead to problems. In particular, *deadlock* and *starvation* are common problems that may be introduced when ensuring mutual exclusion. We explain them using their negation (namely, absence of deadlock and absence of starvation), since these are the properties that we must enforce when solving the mutual exclusion problem.

We start with absence of deadlock.

Definition 7 (Absence of Deadlock). A concurrent program satisfies *absence of deadlock* iff all threads in a concurrent program eventually exit their critical section.

This property may be violated (i.e., the program contains deadlocks) if a thread does not release locks correctly after it has finish executing the critical section. For example, a programmer may forget to add the `unlock()` statement. Another example is a thread crashing while executing the critical section and before executing `unlock()`. In fact, this is the reason why in listing 1.7 we use the `try-finally` idiom for locks in Java. Placing the `unlock()` inside the `finally` block ensures that `unlock()` is always executed; even if the thread crashes while executing the `try` block. Finally, another common source of deadlock is circular lock dependencies. For example if a thread acquires a lock A and then tries to acquire a lock B, which is held by another thread waiting for A.

Absence of starvation is defined as follows.

Definition 8 (Absence of Starvation). A concurrent program satisfies *absence of starvation* iff any threads that are ready to enter the critical section eventually do.

Completely ensuring absence of starvation is very challenging and sometimes impossible. This is due to the non-deterministic behavior of concurrent programs and the scheduler. We can carefully implement a solution to mutual exclusion that contains a mechanism to avoid starvation. But, in many cases, this can only guarantee that all threads eventually reach the enabled state. At that point, it is up to the scheduler to select them for moving to the running state (and execute their critical section). This last step is outside the control of the programmer. Therefore, it is partially outside the control of the programmer to ensure this property. In later chapters, we will see techniques to minimize starvation programmatically.

Finally, we define the properties of a good solution to the mutual exclusion problem.

Definition 9 (Properties of Good Solutions to the Mutual Exclusion Problem). A good solution to the mutual exclusion problem must satisfy the following properties:

1. Ensure mutual exclusion when accessing the critical section
2. Ensure absence of deadlock
3. Ensure (or maximize) absence of starvation